Information Systems Security Project

Στέργιος Λούκας, AEM: 10359

SQL injection fixes

register.php: Δεν κατάφερα να επιτύχω SQL injection, αλλά αυτό παραμένει μια κακή πρακτική, οπότε χρησιμοποίησα prepared statements για να κάνω το INSERT των (username, password) ασφαλές.

```php
// Insert a new user
//comment out this line and make prepared statement
//$sql_query = "INSERT INTO login_users (username,password) VALUES ('{$new_username}','{$new_password}');";
//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query = "INSERT INTO login_users (username,password) VALUES (?,?);";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, "ss", $new_username, $new_password);
$result = mysqli_stmt_execute($statement);
mysqli_stmt_close($statement);
```

login.php: Χρησιμοποίησα το προτεινόμενο payload `xxx' OR 1=1; --` με ένα μη κενό password και κατάφερα να εξάγω όλα τα website/username/password (πίνακας websites). Όπως και πριν, το διόρθωσα χρησιμοποιώντας prepared statement.

```php
// xxx' OR 1=1; -- '
//$sql_query = "SELECT * FROM login_users WHERE username='{$username}' AND password='{$password}';";
//echo $sql_query;
// Check if the credentials are valid
//$result = $conn->query($sql_query);

$sql_query = "SELECT * FROM login_users WHERE username= ? AND password= ? ;";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, "ss", $username, $password);
mysqli_stmt_execute($statement);
$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
```

Dashboard.php: Το ίδιο πρόβλημα όπως προηγουμένως (SQLi) λύθηκε με τον ίδιο τρόπο. Η `mysqli_stmt_get_result()` δεν απαιτείται για λειτουργίες INSERT και DELETE, αλλά δεν προκαλεί προβλήματα στη λειτουργία (έχει μπει σε σχόλια).

```php
// Insert new web site
//$sql_query = "INSERT INTO websites (login_user_id,web_url,web_username,web_password) VALUES " .
            "((SELECT id FROM login_users WHERE username='{$username}'),'{$new_website}','{$new_username}','{$new_password}');";
//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query =  "INSERT INTO websites (login_user_id,web_url,web_username,web_password) VALUES " .
            "((SELECT id FROM login_users WHERE username=?),?,?,?);";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 'ssss', $username, $new_website, $new_username, $new_password);
mysqli_stmt_execute($statement);
//$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
$conn -> close();
```

```php
// Delete selected web site
//$sql_query = "DELETE FROM websites WHERE webid='{$webid}';";
//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query = "DELETE FROM websites WHERE webid=?;";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 's', $webid);
mysqli_stmt_execute($statement);
//$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
$conn -> close();
```

Σε αυτή την περίπτωση η `$result` είναι απαραίτητη επειδή υπάρχει λειτουργία SELECT, οπότε χρησιμοποιούμε την `mysqli_stmt_get_result()`.

```php
// Display list of user's web sites
//$sql_query = "SELECT * FROM websites INNER JOIN login_users ON websites.login_user_id=login_users.id WHERE login_users.username='{$username}';";
//echo $sql_query;
//$result = $conn->query($sql_query);
//echo htmlspecialchars($username);
$sql_query = "SELECT * FROM websites INNER JOIN login_users ON websites.login_user_id=login_users.id WHERE login_users.username=?;";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 's', $username);
mysqli_stmt_execute($statement);
$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
```

Notes.php: Το ίδιο πρόβλημα, η ίδια λύση (SQL injection → prepared statements).

```php
//$sql_query = "INSERT INTO notes (login_user_id, note) ".
//           "VALUES ((SELECT id FROM login_users WHERE username='{$username}'), '{$new_note}')";

//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query = "INSERT INTO notes (login_user_id, note) ".
             "VALUES ((SELECT id FROM login_users WHERE username=?), ?)";

$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 'ss', $username, $new_note);
mysqli_stmt_execute($statement);
//$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
$conn -> close();
```

Εδώ δεν υπάρχει ανάγκη για διόρθωση λόγω έλλειψης άμεσου user input (το άφησα ως είχε).

```php
// Display list of all notes/comments
$sql_query = "SELECT notes.note, login_users.username FROM notes INNER JOIN login_users ON notes.login_user_id=login_users.id;";
//echo $sql_query;
$result = $conn->query($sql_query);

echo "<h3>List of notes/comments</h3>";
```

References: https://www.youtube.com/watch?v=a7iQxCgqpoI

Plain text save fixes

register.php: Εδώ, οι κωδικοί πρόσβασης αποθηκεύονται ως plain text, καθιστώντας τους ευάλωτους σε οποιονδήποτε έχει πρόσβαση στη βάση δεδομένων. Θα χρησιμοποιήσω τη συνάρτηση `getPasswordHash_Hex($username, $password)` από το `test_hash.php` για salt (salt == hashed username) και hash (με sha256) και στη συνέχεια θα αποθηκεύσω τους κωδικούς ως salted hashes, ασφαλίζοντάς τους σωστά.

Αρχικά, έβαλα σε σχόλια τα παραδείγματα στα `test_encrypt.php` και `test_hash.php` για να συμπεριλάβω τα αρχεία και να χρησιμοποιήσω τις συναρτήσεις: `hash_function` στο `register.php`.

```php
//include('includes/hash_function.php');
function getPasswordHash_Hex($username, $password) {
    // Compute hash of salted-password (and salt) from username and password (in hex format)
    $salt = hash('sha256', $username);  // Compute salt as the hash of the username
    $saltedPwd = $salt . $password;     // Get a salted password by combining salt and password
    $hashedPwd = hash('sha256', $saltedPwd);    // Hash the salted password using SHA-256
    // Return the password hash and the salt
    return [
        'hash' => $hashedPwd,
        'salt' => $salt
    ];
}
```

```php
include('test_hash.php');
```

Στη συνέχεια χρησιμοποίησα τη συνάρτηση για να αποθηκεύσω το hashed password.

```php
    // Get user submitted information
    $new_username = trim($_POST['new_username']);
    $new_password = trim($_POST['new_password']);
    $getHasedPwd = getPasswordHash_Hex($new_username, $new_password);
    $new_password = $getHasedPwd['hash']; //only save the hashed password and not the 'salt'
    mysqli_report(MYSQLI_REPORT_OFF);  // disable exceptions
```

Μετά την εφαρμογή των αλλαγών, εγγράφηκαν επιτυχώς νέοι χρήστες με hashed passwords.

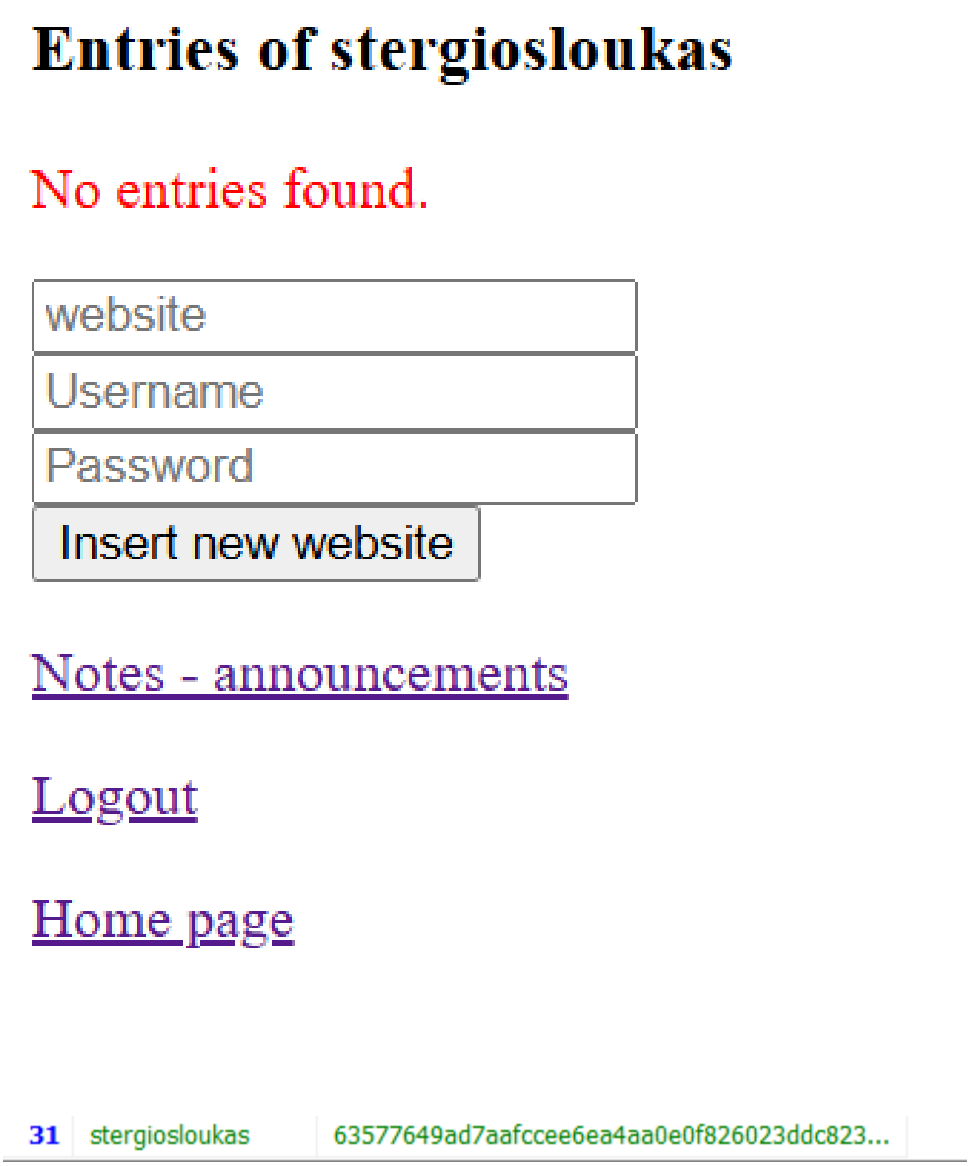
u45	c629dee600a45132f5f430b9db399c29618e8665...
u50	2a990e084bfa9d43d8d16b52556a99cb13073eb...
sdadsadsa	11633539c2aa552f48144dbcbb0a21cd2254e7a...
stergiosloukas	63577649ad7aafccee6ea4aa0e0f826023ddc823...

Μετά από αυτό έπρεπε να ενημερώσω το `login.php` ανάλογα, ώστε να ακολουθεί τη λογική του hashing για την επαλήθευση του user input/login credentials. Συμπεριέλαβα το αρχείο της συνάρτησης και εδώ και ενημέρωσα τη λογική.

```php
<?php
include('test_hash.php');
```

```php
else {
    // Get user submitted information
    $username = trim($_POST['username']);
    $password = trim($_POST['password']);
    $getHasedPwd = getPasswordHash_Hex($username, $password);
    $password = $getHasedPwd['hash'];
```

Μετά την εφαρμογή των αλλαγών, συνδέθηκα επιτυχώς στο σύστημα ως χρήστης με αποθηκευμένο hashed password.



Website Passwords: Ένα άλλο ζήτημα είναι ότι οι κωδικοί πρόσβασης των ιστότοπων (website passwords) δεν εισάγονται ως κρυπτογραφημένες τιμές, εκθέτοντάς τους και καθιστώντας τους εύκολη λεία για κάποιον με πρόσβαση στη βάση δεδομένων.

login.php: Αρχικά εξάγω το session key, διότι ενώ το username είναι διαθέσιμο στη σελίδα του dashboard, το password είναι διαθέσιμο μόνο στο login.

```php
<?php
include('test_encrypt.php');
```

```php
//key derivation to encrypt/decrypt data in dashboard.php
$key = $_SESSION['encryption_key'] = deriveEncryptionKey($username, $password);
```

dashboard.php: Στη συνέχεια κρυπτογραφώ το web password χρησιμοποιώντας το session key πριν το εισάγω στη βάση δεδομένων.

```php
<?php
include('test_encrypt.php');
```

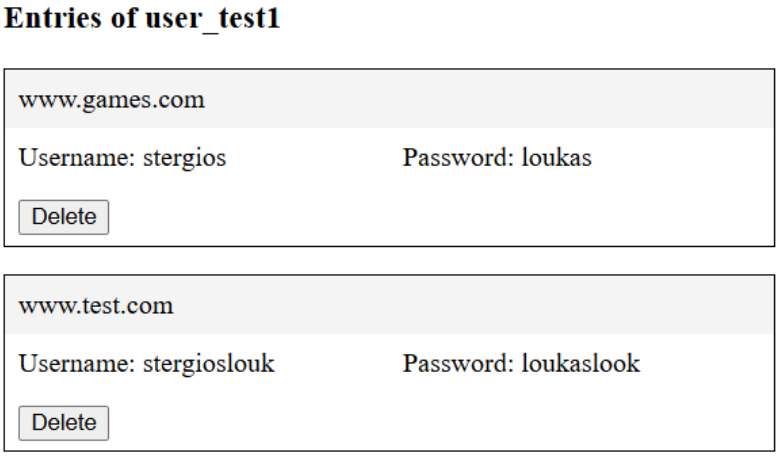
```php
//password encryption using the derived session key from login.php
$key = $_SESSION['encryption_key'];
$encrypted_password = encryptData($new_password, $key);
$new_password = $encrypted_password;
```

Και όταν το password πρέπει να εμφανιστεί, αποκρυπτογραφείται χρησιμοποιώντας την αντίστροφη λογική, με το ίδιο session key (επειδή το username και το password παραμένουν τα ίδια).

```php
if (!empty($result) && $result->num_rows >= 1) {
    while ($row = $result -> fetch_assoc()) {
        //decrypt the password before displaying
        $key = $_SESSION['encryption_key'];
        $row["web_password"] = decryptData($row["web_password"], $key);
```

Μετά την εφαρμογή των αλλαγών, ήμουν σε θέση να αποθηκεύσω ένα κρυπτογραφημένο web password και να το βλέπω κανονικά (decrypted) μέσα στην εφαρμογή.

23	34	www.games.com	stergios	+DQHV33t93BaIXgThmDLtWNIdG0Jmsyyp51iXC8VtIfigw==
24	34	www.test.com	stergioslouk	SkLIMR7hqbZqeAIhK5qz/vjnUU+S3z5xsk7INGcgIlg/8ly5nKI=



notes.php: Τα notes θα μπορούσαν επίσης να κρυπτογραφηθούν/αποκρυπτογραφηθούν, αλλά δεν υπάρχει ανάγκη γι' αυτό, καθώς είναι δημόσια για όλους τους συνδεδεμένους χρήστες.

References:

- https://www.youtube.com/watch?v=zt8Cocdy15c
- `passman/test_hash.php` → function `getPasswordHash_Hex($username, $password)`
- `passman/test_encrypt.php` → function `deriveEncryptionKey($username, $password)`
- `passman/test_encrypt.php` → function `encryptData($data, $key)`
- `passman/test_encrypt.php` → function `decryptData($data, $key)`

Cross Site Scripting (XSS) fixes

dashboard.php: Εμφανίζει μη-sanitized δεδομένα αποθηκευμένα από τον χρήστη, καθιστώντας την εφαρμογή ευάλωτη σε stored XSS attacks. Ωστόσο, μόνο ο συνδεδεμένος χρήστης μπορεί να δει αυτά τα δεδομένα, οπότε μια επίθεση είναι αδύνατη, εκτός αν κάνει self-XSS. Για κάθε ενδεχόμενο, έκανα sanitize και αυτό το output, χρησιμοποιώντας την `htmlspecialchars()`.

```
(!empty($result) && $result->num_rows >= 1) {
 while ($row = $result -> fetch_assoc()) {
    //decrypt the password before displaying
    $key = $_SESSION['encryption_key'];
    $row["web_password"] = decryptData($row["web_password"], $key);
    echo "<table border=0>";
    echo    "<tr style='background-color: #f4f4f4;'><td colspan=2>" . htmlspecialchars($row["web_url"]) . "</td></tr>" .
            "<tr><td>Username: " . htmlspecialchars($row["web_username"]) . "</td><td>Password: " . htmlspecialchars($row["web_password"]) . "</td></tr>";

    echo    "<tr><td><form method='POST' style='height: 3px'>" .
            "<input type='hidden' name='websiteid' value='" . htmlspecialchars($row["webid"]) . "'>" .
            "<button type='submit' name='delete_website'>Delete</button></form></td></tr>";

    echo    "<tr><td colspan=2 style=height: 20px;></td></tr>";
    echo "</table><p/>";
```

notes.php: Από την άλλη πλευρά, το `notes.php` αποθηκεύει δεδομένα όλων των χρηστών και τα εμφανίζει χωρίς sanitization, οπότε είναι επιτακτικό να καθαρίζονται πρώτα, ώστε ο κακόβουλος κώδικας να μην εκτελεστεί στην πλευρά κάποιου άλλου χρήστη. Επομένως, επεξεργάστηκα το display section για να κάνω sanitize το output χρησιμοποιώντας την `htmlspecialchars()` πριν την εμφάνιση των notes.

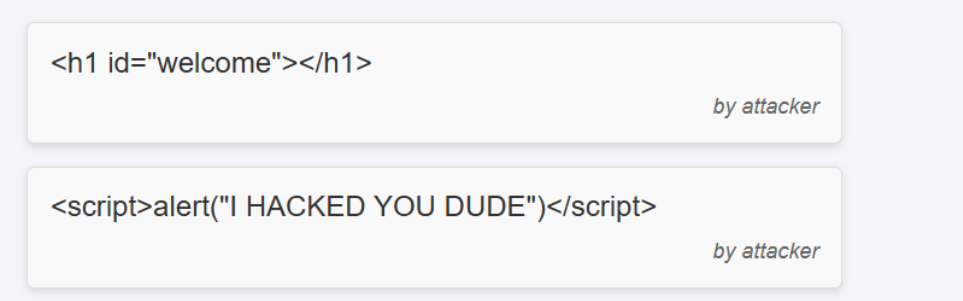
```
if (!empty($result) && $result->num_rows >= 1) {
    while ($row = $result -> fetch_assoc()) {
        echo "<div class='note'>";
        //echo  "<div class='note-content'>" . $row["note"] . "</div>";
        //echo  "<div class='note-signature'> by " . $row["username"] . "</div>";
        //Sanitization against XSS
        echo "<div class='note-content'>" . htmlspecialchars($row["note"]) . "</div>";
        echo "<div class='note-signature'> by " . htmlspecialchars($row["username"]) . "</div>";
        echo "</div>";
    }
```

Testing and Confirmation: Για να επιβεβαιώσω τη διόρθωση, έκανα πλήρες reset της βάσης δεδομένων (διέγραψα όλα τα δεδομένα από `login_users`, `notes`, `websites`) και δημιούργησα δύο λογαριασμούς, *attacker* και *defender*, για να ελέγξω τις ευπάθειες XSS.

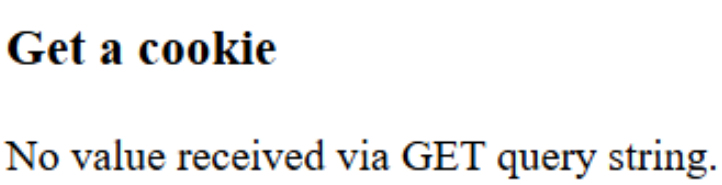
pwd_mgr.login_users: 2 rows total (exact)

#	1 id	2 username	3 password
1	1	attacker	ae051a9330432e3ac12d91b2af0db8b542a00af7...
2	2	defender	2291461bc5f5c46259e8223827460efa0a55979f...

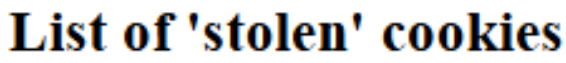
Στη συνέχεια, δημοσίευσα ορισμένα κακόβουλα XSS payloads στα notes ως attacker.



Μετά την είσοδο ως defender και την επίσκεψη στη σελίδα των notes, δεν εμφανίστηκε κανένα alert στο browser μου. Επίσης, πρόσθεσα μια καταχώριση website/username/password στο λογαριασμό μου ως defender. Έλεγξα επίσης τις σελίδες από την πλευρά του hacker μετά τη σύνδεση και, όπως αναμενόταν, δεν κλάπηκε κανένα cookie.



Το αρχείο `Stolencookies.txt` παραμένει επίσης κενό.



Και η σελίδα `usecookie` στερείται επίσης του cookie που απαιτείται για το hack στη σελίδα του defender.

Test of using a stolen cookie

Bypassing authentication and impersonating another user by using a stolen cookie
Session ID is set to: **PHPSESSID=**
Username: undefined (session variable expired)
Logged in flag: undefined (session variable expired)

If all above session parameters are defined, try accessing the dashboard

List cookies

Μετάβαση από HTTP σε HTTPS: Αρχικά δημιούργησα ένα self-signed SSL Certificate. Στη συνέχεια, ενεργοποίησα το SSL στον Apache web server και το παραμέτρησα με το self-signed πιστοποιητικό (`server.crt` και `server.key`). Η θύρα (Port) 80 ακούει σε HTTP και η θύρα 443 ακούει σε HTTPS μετά τη νέα παραμετροποίηση.

```
PS C:\xampp\htdocs\passman> Test-Path "c:\xampp\apache\conf\ssl.crt\server.crt"
True
PS C:\xampp\htdocs\passman> Test-Path "c:\xampp\apache\conf\ssl.key\server.key"
True
```

<script>alert("I HACKED YOU DUDE")</script>

by attacker

I guess you didn't... hahaha

by defender

The end.